

Generalized Analogical Reasoning Through Latent Geometric Alignment

Scope

The current artifact is a cognitive idea engine that can generate new ideas and hypotheses through creating analogical connections between knowledge domains. It is made up of several modules/mechanisms that work together to achieve specific cognitive tasks.

The system operates by projecting a user's query (corresponding to one domain of knowledge) into a shared latent space (used for all domains). This enables the creation of meaningful relationships between different domains and once created, can be searched for the connecting path to the solution to the query.

A working memory module that takes after the role of working memory in the human brain is used to store any latent vector information found relevant during the travel process. This stored information includes intermediate states to getting to the solution as well as any contextual information encountered. Due to the use of working memory, once a useful solution is reached, the system is able to produce the explainable steps it took to get there - taken from the reasoning chain in working memory history. A salience module within the system works to direct the switch between the default mode process (for new idea generation and creative exploration) and executive control process (for goal-directed reasoning, hypothesis refinement and evaluation) depending on the recognized query type. This structure is also modelled after the brain.

The system architecture provides a means for an artificial cognitive system capable of **creative reasoning, cross-domain hypothesis generation, and self-improving analogical thought**, representing a significant step toward artificial general intelligence.

Background of the Artifact

The artifact relates to artificial intelligence and is a system capable of cross domain reasoning to generate hypotheses and new ideas which is enabled through analogical alignment and latent space representation.

The current state of AI harbors many strengths such as complex pattern recognition, natural language generation, and generating new content multi-modally. This has led to AI being applied in a variety of different fields and problems, and in many cases artificial intelligence has excelled in solving many problems more so than the traditional methods. However, these systems are usually trained on large, domain-specific datasets and lack the capability to generalize across different domains without retraining.

Existing approaches to transfer learning and multi-task learning provide partial solutions to these problems, but are limited in scope. These systems often require fine-tuning on target domains and struggle to extend their reasoning to completely novel problem spaces. Similarly,

current embedding-based methods capture latent representations of concepts but do not inherently support analogical projection across different domains.

Cognitive science research suggests that human intelligence relies heavily on analogy and the recognition of shared relational structure across different contexts for learning, understanding, reasoning, and creativity - all of which are based on analogical reasoning. Despite advances in machine learning, few computational models have successfully operationalized analogy-making as a general mechanism for reasoning across domains. Existing analogy models either rely on hard-coded symbolic structures - which do not scale - or deep learning methods that must fit only to their training domain, which do not generalize.

Therefore, there is a need for an artificial intelligence system that can reliably conduct analogical reasoning across different domains to effectively enable zero-shot reasoning, for the further purpose of explainable creative reasoning and hypothesis generation

Traditional artificial intelligence systems are typically trained within narrow task domains, limiting their ability to generalize or reason analogically across disciplines. Despite major advances in deep learning, current architectures often fail to capture the deeper structural relationships that unify knowledge representations across different cognitive domains such as physics, biology, and economics.

One of the main theoretical assumptions that the system takes advantage of is that all cognitive domains share underlying structure, at least when it comes to analogy. This means, for example, that a pair of concepts in the domain of biology having some relationship x between them, there exists a pair of concepts with that same relationship x in the domain of economics, for example. And how the relationship x impacts any concept in one domain is enough information to determine the nature of its impact in another.

Upon taking this assumption, the creation of analogy across domains becomes possible, because a network, much like the human mind, that is designed to see patterns in the structure of knowledge can identify which concepts in another domain the current problem corresponds to, which is exactly the mechanism that makes the system possible. This approach then, should only require the training and learning of one domain in order to generalize to others, since analogical projection of a new problem into a familiar space can occur, and then reasoning occurs within the familiar space, which serves as insight into the new problem.

Given this, cross-domain generalization becomes possible once the corresponding concepts are found, and the following reasoning only requires some means of translation back to the language of the original domain of the problem for a user to understand the output.

The present artifact builds on this assumption by providing a computational system capable of learning and utilizing these underlying structural correspondences. Through projection into a shared latent space and traversal along meaningful conceptual axes, the system enables general-purpose reasoning and cross-domain hypothesis generation — functions

traditionally associated with human-like intelligence. Such a system would enable the discovery of structural correspondences across domains, support the inference of missing concepts, and facilitate the generation of novel hypotheses and ideas, thereby advancing the field toward more general and creative forms of machine intelligence.

Summary of the System Components

Detailed Description of the Artifact

Below is a detailed description of the operation of all the components of the system and how they interact, as well as why some architectural choices were made. A later section describes key mechanisms and processes in greater detail.

Deep Probabilistic VAE:

In one embodiment, the system includes a deep probabilistic variational autoencoder (VAE) trained on some Domain A's data (represented by E_A) that encodes that data into a **probabilistic** latent space. This means every concept is encoded as a cloud of possible positions.

In certain implementations, the encoder outputs a Gaussian distribution with a learned mean (μ_x) and variance (Σ_x) for each input

$$E_A(x) = N(\mu_x, \Sigma_x), \quad z \sim q_\phi(z|x)$$

Reasoning:

- The system requires a well structured detailed latent space within which reasoning can occur, which the probabilistic nature of the VAE provides. The probabilistic loss function of a VAE (the KL-divergence term) ensures the latent space is smooth and continuous, which is the entire objective of using this component. That's what makes *latent traversal* meaningful — small changes in z correspond to semantically coherent shifts (like moving from “*mass*” → “*weight*” → “*force*”). The encoder lays the groundwork and initial setup for the rest of the system by providing the latent space.
- Geometrically - Because the latent variables represent probability distributions, the *distance* between two points reflects *how likely their underlying meanings overlap*. This

builds a **semantic topology** that encodes both certainty and analogy within the latent space of Domain A.

- Creatively - When structural perturbation is performed, a probabilistic latent model ensures that this small change remains *on-manifold* — i.e., the new point still represents a valid concept. This kind of fuzzy matching offers controlled creativity but also controlled hypothesis generation, the mechanisms of which will be explored in greater detail in a subsequent section.

Thus, the encoder establishes not only the **probabilistic topology** of the Knowledge Space but also its **structured relational geometry**, forming the substrate on which all subsequent reasoning, traversal, and idea-generation operations depend.

In some embodiments, the **input** to the encoder comprises vector embeddings of textual or symbolic data from Domain A, while the **output** comprises the latent representation or “knowledge space” used for reasoning.

Knowledge Space

The formal name for the latent space of Domain A created by the encoder, this is where projection of the user query occurs, and the singular location where traversing between concepts for idea and hypothesis generation occurs. It has **probabilistic geometry** - meaning any point will not be dealt with as one but is identified as a mean of a cloud of points (to describe the center) and covariance (to describe how spread out the cloud is). The greater the uncertainty about what the concept is the larger the covariance of the cloud, while the stronger the analogical equivalent in Domain A the smaller the cloud.

Within the Knowledge Space:

- **Directions** correspond to *relationships* (e.g., cause→effect, increase→decrease)
- **Distances** correspond to *conceptual similarity*
- **Regions** represent *concept clusters* or knowledge domains
- **Traversal** corresponds to *reasoning paths*

To enable structured manipulation of concepts, the encoder additionally learns a set of **latent basis vectors or “learned axes”**, denoted ($A = \{a_1, a_2, \dots, a_k\}$).

Each learned axis represents an interpretable relational direction within the latent manifold, emerging from unsupervised discovery of consistent transformations such as:

- whole ↔ part
- cause ↔ effect
- abstract ↔ concrete
- means ↔ goal
- agent ↔ object

These axes serve as **semantic coordinate directions** of the latent geometry.

Subsequent reasoning operations—including traversal, analogy, and structural perturbation—can therefore operate not merely on arbitrary vectors but along **meaningful, learned relational dimensions**.

The geometry of these axes defines the **operational topology** of the Knowledge Space:

- Learned axes encode invariant relationships that generalize across domains, which are instrumental in traversal and generating new ideas.
- Vector composition and projection along these axes yield interpretable analogical reasoning paths.
- Traversal and creative perturbation modules later reference these axes to generate coherent yet novel concept combinations.

Reasoning: As a model to the brain's own knowledge space. It is a **geometric representation** of the brain's own knowledge space, which can be maneuvered through to reach different concepts using the known relations between them, which were encoded meaningfully and semantically into the space by the encoder.

Projection Network:

A neural network trained on meta-analogies for Domain A - it learns analogical structure and learns to properly align the projection of the query of another domain (Domain B) into the latent space. Query projection requires **dual-projection**, which includes the entire query itself as well as multiple other vectors that correspond to the other concepts and relationships found in the query. Due to the probabilistic nature of it actually outputs **parameters of a distribution** from which a **sample** can be drawn. The projection network in your system outputs *distributions* —

not fixed vectors — corresponding to *probable analogical positions* in the encoded domain’s latent space.

This lets the two thinking modes later *sample*, *traverse*, and *refine* ideas probabilistically — making the system flexible, creative, and human-like in its uncertainty handling that is built into the system probabilistically.

Let Q_b be decomposed into its semantic components $\{c_1, c_2, \dots, c_n\}$ for multi vector projection.

For example:

Query: “What is the brain’s equivalent of a computer’s processor?”

Components: c_1 =brain, c_2 =equivalent, c_3 =computer’s processor

The Projection Network f_ψ produces two outputs:

1. **Global projection:** $z_B^{\text{global}} = f_\psi(Q_B)$ for the entire query
2. **Local projections:** $z_{B,k}^{\text{local}} = f_\psi(c_k(Q_B))$ for each sub-component

In one embodiment, this could be the analogical alignment loss:

$$L_{\text{proj}} = \sum_k d(f_\psi(c_k(Q_B)), N(z_{A,\text{ref},k})) + \alpha d(f_\psi(Q_B), C(z_{A,\text{ref}}))$$

Here, $(\mathcal{N})$ represents neighborhood mappings around reference concepts in Domain A, while $(\mathcal{C})$ denotes centroids of conceptual clusters.

(1) Local alignment term

$$\sum_k d(f_\psi(c_k(Q_B)), N(z_{A,\text{ref},k}))$$

- Each projected component from domain B (e.g., “processor”) is forced to **land near** a corresponding neighborhood $N(z_{a,\text{ref},k})$ in Domain A’s space (e.g., “neurons” or “frontal cortex” in biology).
- This enforces **part-to-part analogical correspondence**.

(2) Global alignment term

$$\alpha d(f_\psi(Q_B), C(z_{A,\text{ref}}))$$

- Ensures the entire query embedding (global meaning) lies near the centroid (average concept) of the relevant region in A's latent space.
- This captures overall semantic or relational similarity.

Reasoning:

When humans are creating analogies between domains they usually need to align the concepts of one domain with those of another, so that the new alignment can tell them about the relationships between all the concepts. They also require the right “context”, which is exactly the role of the projection network - to learn how to align queries based on the relationships of the concepts they describe in a general way that can work for any domain's query, and project them into the latent space in the right context ‘space’. Being in the right context space means the Domain B projections landed near the most semantically similar Domain A concepts.

In some embodiments, the projection network takes as **input** the embeddings corresponding to the user's query and **outputs** distributions in Domain A's latent geometry representing analogical correspondences for both global and local structures.

Working Memory

The working memory is a bank where relevant vectors encountered during knowledge space navigation when answering a query are recorded. The order of vectors within working memory offers a kind of traceable path through which reasoning can be understood when outputted by the decoder. As the working memory interacts with the other modules, especially the verifier module, vectors or pathways that were recorded may be erased due to low plausibility, keeping the real path intact.

The working memory maintains contextual reasoning state — a history of traversed latent points, coherence scores, and semantic transitions.

In some embodiments, each memory entry is the following tuple:

$$M_t = \{z_t, \Delta z_t, mode_t, novelty_t, coherence_t\}$$

where t is the time step or iterative index during reasoning, and z is some point/concept that was traversed to at time t.

Δz_t = the **change in the latent state** from step t to step t+1

This tells us *how the system's concept evolved*:

- In creative mode, Δz might push toward novel regions (high novelty gradient).
- In goal-directed mode, Δz moves toward a target (reducing semantic distance).

mode $\square$ = which reasoning mode was ongoing at step t :

- "DM" = Default Mode (creative/exploratory)
- "EC" = Executive Control (focused/problem-solving)

This helps later explain *why* the system made a certain reasoning step — e.g. “This jump was exploratory” vs “This move was goal-directed.”

novelty $\square$ = how unusual or unexplored the current concept z_t is within the conceptual manifold.

$$N_t = ||z_t - z_{\text{prev}}||$$

It measures *creative distance* from familiar regions:

- High novelty $\rightarrow$ idea is far from known examples (potentially creative).
- Low novelty $\rightarrow$ idea is well-trodden (more conventional reasoning).

It can be computed using information density or KL-divergence from data distribution $p_{\text{data}}(z)$.

coherence $\square$ = how internally consistent or semantically well-formed the concept at z_t is.

$$C_t = \cos(z_t, \text{neighbors}(z_t))$$

If the system’s reasoning becomes inconsistent, this drops — triggering backtracking or correction.

Dynamic Updating and Pruning

As the system traverses the knowledge space, the working memory dynamically updates and prunes low-relevance or implausible entries based on the **feedback** from the verifying mechanism.

If coherence falls below a threshold τ_{Cmin} or novelty exceeds a noise ceiling τ_{Nmax} , the entry is removed:

$$M_t \leftarrow M_t \setminus \{M_i \mid C_i < \tau_{\text{Cmin}} \text{ or } N_i > \tau_{\text{Nmax}}\}$$

This ensures that only semantically plausible, connected reasoning chains remain in memory.

Reasoning: The brain uses a working memory to hold the context of any problem and when thinking for a solution fills it with relevant information that could potentially help solve the problem while it searches for the solution. And as some ideas are deemed unusable, they are free to be forgotten about, hence the flagging for removal in the working memory module of the system. Adding a working memory also allows for longer chains of “thought” or traversal since the working memory can theoretically hold more than some brains can at a time. Used both for iterative reasoning and for post-hoc explanation generation.

Input: Relevant vectors and positions are appended to it throughout the traversal process.

Output at the end of process: The reasoning path that led to the solution(s), enabling explainable output.

Saliency Network:

The mechanism that switches between the Default Exploration Mode and the Executive Control Mode, which are two different ways of approaching finding the answer to the query, There are many ways the system may be used, but two are to ask to generate new ideas and hypothesis generation. For any given open-ended query, at some point the modes switch.

In some embodiments, initially, the query is evaluated to check whether a relation vector is present (described in a later section). And if one is detected, traversal starts being governed by the Executive Control Mode initially. If not, the Default Mode starts initially.

In some other embodiments, some mathematical threshold is used to determine if the query is more open-ended or goal-directed, and based on that the saliency network selects either Default Mode or Executive Control Mode.

The Saliency Network monitors the ongoing traversal and **dynamically regulates switching** between the Default Mode (DM) and Executive Control Mode (EC), which represent two complementary approaches to exploring and reasoning over the knowledge latent space. The system may be used in various contexts, including open-ended idea generation and hypothesis formation.

- **Initial Mode Selection:**

In some embodiments, the query is first evaluated for the presence of a relational vector. If one is detected, traversal begins in **EC Mode**; if not, traversal begins in **DM Mode**.

In other embodiments, a threshold-based evaluation of the query's openness versus goal-directedness is used to select the initial mode.

- **Continuous Monitoring:**

The Saliency Network continuously assesses **coherence (C)** and **novelty (N)** metrics derived from Working Memory. Unlike a one-time switch, this monitoring allows **adaptive back-and-forth switching**, enabling the system to return to previously explored DM paths or refine EC candidates as needed.

It continuously evaluates **coherence (C)** and **novelty (N)** from the working memory entries.

Mode switching rules:

$$\begin{aligned} \text{If } C_t < \tau_C \text{ and } N_t > \tau_N &\Rightarrow \text{stay in DM} \\ \text{If } C_t > \tau_C \text{ and } N_t < 0 &\Rightarrow \text{switch to EC} \end{aligned}$$

Switching is not final; the system may return to DM if EC stagnates or novelty drops below desired levels.

Mode Descriptions:

- **Default Mode (DM):** Exploratory traversal that seeks novel connections between latent concepts without strict enforcement of relational constraints. Looser guidelines allow creative idea generation while monitoring coherence to avoid nonsensical outputs.
- **Executive Control Mode (EC):** Goal-directed traversal emphasizing relational alignment and semantic coherence with the query. Outputs are closely monitored by the Verifier Module for relevance and plausibility.

Reasoning Analogy:

This mirrors human cognition: the brain dynamically balances between exploratory, creative thinking (DM) and focused, goal-directed reasoning (EC). The Saliency Network ensures that novelty is maximized without sacrificing coherence, enabling the system to iteratively refine ideas while retaining previously explored possibilities.

Decoder:

The **decoder** translates the final reasoning trace or latent representation into an interpretable form. In some embodiments, the decoder reconstructs symbolic or linguistic expressions that represent both the outcome and the reasoning process.

Reasoning: The decoder is the equivalent of a human completing an analogy by reasoning in another domain than initially, and translating the reasoning back into the initial domain's terms.

The **input** comprises a working memory output, and the **output** includes an explainable reasoning path.

Detailed Description of Mechanisms and Processes

- Place where I explain where the mechanisms of algs work with the main components and which are involved in the mechanisms, etc.
- explains better how the components work too

The Mechanisms and Flow of Traversal

1. Relation Extraction

When humans reason about a problem or analogy, they consider both objects (concepts) and the relations that connect them. For example, when considering “Can biological evolution principles improve machine learning optimization?”, the mind aligns two conceptual entities — “evolution” and “machine learning optimization” — through a relational connector, “improve.”

In this system, the relational vector represents the semantic transformation between two concepts — the “direction of meaning” connecting them (e.g., algorithm → optimization encodes “process that improves performance”). During reasoning, traversal occurs along and around this relational vector within the latent space: it defines the primary axis of motion (distinct from the learned axes in the Knowledge Space) and guides the system toward analogical equivalents in the target domain.

Traversal is **dynamic**, adjusting using coherence and novelty gradients to remain in meaningful regions of the curved manifold. The relational vector acts as a semantic compass, exploring trajectories that preserve analogical structure while testing for coherence.

When a query is received, the Projection Network decomposes it into:

- **Entities:** key concepts or domains being compared
- **Relations:** words or phrases expressing interactions (e.g., “improve,” “affect,” “optimize”)

Optionally, a lightweight dependency or semantic parser isolates these components automatically. For a query (Q_B):

[
 $Q_B = (E_1, R, E_2)$
]

where (E_1) and (E_2) are entities and (R) is the relation term.

Example:

“Can biological evolution principles improve machine learning optimization?”

[
 $E_1 = \text{‘biological evolution’}, \quad R = \text{‘improve’}, \quad E_2 = \text{‘machine learning optimization’}$
]

2. Relational Projection into Latent Space

The Projection Network maps components into the Knowledge Space of Domain A:

[
 $z_{\{E_1\}} = E_A(E_1), \quad z_{\{E_2\}} = E_A(E_2), \quad r_{\text{term}} = P_{\{B \rightarrow A\}}(R)$
]

This produces two concept embeddings ($(z_{\{E_1\}}, z_{\{E_2\}})$) and one relation embedding ((r_{term})), each represented probabilistically:

[
 $z_{\{E_i\}} \sim \mathcal{N}(\mu_i, \Sigma_i), \quad r_{\text{term}} \sim \mathcal{N}(\mu_R, \Sigma_R)$
]

allowing uncertainty to influence relational reasoning (mirroring biological “conceptual fuzziness”).

3. Constructing the Explicit Relation Vector

The relation vector blends the structural difference between entities and the semantic direction implied by the relation word:

$$\begin{aligned} &[\\ &r_{\text{cand}} = z_{E_2} - z_{E_1} \\ &] \\ &[\\ &a_{\text{rel}} = \text{normalize}(\big((1 - \alpha) r_{\text{term}} + \alpha r_{\text{cand}}\big)) \\ &] \end{aligned}$$

Where:

- (a_{rel}) is the explicit relation vector
- $(0 < \alpha < 1)$ balances data-driven vs. linguistic signals
- $\text{normalize}()$ scales the vector to unit length

This vector encodes relational intent as an operator in latent space rather than a discrete token.

4. Relation-Guided Traversal

At each step (t), the concept position (z_t) is updated:

$$\begin{aligned} &[\\ &z_{t+1} = z_t + \eta \big(\lambda_1 a_{\text{rel}} + \lambda_2 \nabla_z C(z_t) + \lambda_3 \\ &\nabla_z N(z_t) + \lambda_4 P(z_t) \big) \\ &] \end{aligned}$$

Where:

- (η) = step size
- $(\lambda_1, \lambda_2, \lambda_3, \lambda_4)$ = weights for relation, coherence, novelty, and perturbation

Traversal Behavior:

- Moves along the relational axis when present

- Maintains coherence
- Encourages exploration via novelty and structural perturbation

Mode-Specific Weighting:

- **DM:** emphasizes (λ_3) (novelty) and (λ_4) (structural perturbation)
- **EC:** emphasizes (λ_1) (relation) and (λ_2) (coherence), de-emphasizing novelty

Structural Perturbation: DM introduces controlled deviations along learned axes ($A = \{a_1, a_2, \dots, a_k\}$):

$$z' = z_t + \sum_i \tau_i a_i + \epsilon R(z_t)$$

- $(\tau_i \sim \mathcal{N}(0, \sigma^2))$
- $(R(z_t))$ = small random rotation in tangent space
- (ϵ) = exploration magnitude

5. Termination and Verification

Traversal continues until:

- **Goal achievement:** $(\cos(z_t, z_{E_2}) > \tau_G)$
- **Novelty plateau:** $(|\nabla_t N(z_t)| < \epsilon_N)$
- **Maximum step limit reached**

Final latent state (z^*) is decoded by the Decoder into interpretable hypotheses.

6. Sample Example

Query: “Can biological evolution principles improve machine learning optimization?”

1. Projection:

- $(z_{E_1}) \rightarrow$ evolution region
- $(z_{E_2}) \rightarrow$ optimization region
- $(r_{\text{term}}) \rightarrow$ “improve” embedding

2. Relation Axis:

$$\begin{bmatrix} a_{\text{rel}} = \text{normalize}((1-\alpha) r_{\text{term}} + \alpha (z_{E_2} - z_{E_1})) \end{bmatrix}$$

3. Traversal:

$$\begin{bmatrix} z_{\text{new}} = z_{E_2} + \eta a_{\text{rel}} \quad (\text{plus DM perturbations if applicable}) \end{bmatrix}$$

4. Decoding Outputs:

- “Evolutionary algorithms for model optimization”
- “Population-based meta-learning strategies”
- “Genetic selection in hyperparameter search”

7. Learned Axes and Relation Families

Relation vectors cluster into families forming a basis of learned axes, e.g.:

- “cause $\rightarrow$ effect”
- “part $\rightarrow$ whole”

- “increase → decrease”
- “optimize → improve”

New relations can be expressed as linear combinations:

$$[a_{\{\text{rel,new}\}} = \sum_i w_i a_{\{\text{rel}\},i}]$$

8. Interaction with Modules

Module	Interaction
Encoder / Knowledge Space	Provides latent geometry and learned axes
Projection Network	Identifies entities and relations; outputs embeddings
Working Memory	Stores traversal paths, DM perturbations, coherence/novelty metrics
Verifier Module	Confirms relation-consistent reasoning, prunes incoherence
Salience Network	Monitors coherence and novelty; triggers dynamic DM ↔ EC switching
Decoder	Generates human-readable hypotheses from latent vectors

9. Output

The system outputs latent vectors (z^*) representing analogical transformations consistent with the query. Decoding produces semantically coherent and novel ideas (e.g., applying “evolutionary selection” to “machine learning optimization”).

Multi-relation reasoning: Multiple relation vectors can be composed for complex queries (e.g., “affects” + “over time” = temporal causation).

Mode behavior:

- **Default Mode:** searches the manifold for new connections when no explicit relationship is given.
- **Executive Control:** refines or confirms known relational paths.

Here’s the updated **traversal sections** rewritten to fully integrate **dynamic DM ↔ EC switching** with salience monitoring while keeping the patent-style formalism:

Traversal in Executive Control Mode (EC) with Dynamic Switching

In Executive Control Mode, the system emphasizes **goal-directed, relation-consistent reasoning**, but under the hybrid design, traversal can **dynamically switch back to DM** if novelty drops or local minima are reached.

1. Initial Position

Traversal begins at:

- An entity embedding ($z_0 = z_{\{E_1\}}$) from the query, or
- A candidate vector from prior DM exploration

The relation vector (a_{rel}) is derived from the query’s relational terms (e.g., “improve,” “optimize”) via the Projection Network:

$$\begin{aligned}
& [\\
& r_{\text{term}} = P_{\{B \rightarrow A\}}(R) \\
&] \\
& [\\
& a_{\text{rel}} = \text{normalize}((1 - \alpha) r_{\text{term}} + \alpha (z_{E_2} - z_{E_1})) \\
&]
\end{aligned}$$

Here:

- (r_{term}) = linguistic projection of the relation
 - $(z_{E_2} - z_{E_1})$ = optional structural difference between entities
 - (α) = weight controlling structural vs. linguistic signal
-

2. Update Rule

At each step (t), the concept vector (z_t) is updated:

$$\begin{aligned}
& [\\
& z_{t+1} = z_t + \eta \left(\lambda_1 a_{\text{rel}} + \lambda_2 \nabla_z C(z_t) + \lambda_3 \nabla_z N(z_t) + \lambda_4 P(z_t) \right) \\
&]
\end{aligned}$$

Where:

- (λ_1) = relation weight (dominant in EC)
 - (λ_2) = coherence weight
 - (λ_3) = novelty weight (de-emphasized but monitored)
 - (λ_4) = structural perturbation weight (small, for exploration)
 - $(P(z_t))$ = structural perturbation operator
-

3. Dynamic Traversal Behavior

1. Primary EC behavior:

- Follow (a_{rel}) toward the target concept (z_{E_2})
- Maintain high coherence via ($\nabla_z C(z_t)$)
- Monitor novelty without prioritizing it

2. Salience-based switching:

- If novelty drops below threshold (τ_N) or traversal stagnates, the **Salience Network triggers a switch to DM**
- DM then explores alternative directions in the local manifold using structural perturbations
- Working Memory ensures previously explored DM paths are not lost and can be revisited

3. Return to EC:

- Once coherence improves or a promising candidate aligns with (a_{rel}), traversal switches back to EC
- This **dynamic back-and-forth continues** until halting conditions are met

4. Termination Criteria

Traversal continues until:

- Goal achievement: ($\cos(z_t, z_{E_2}) > \tau_G$)
 - Maximum step limit reached
 - Working Memory / Verifier detects that further exploration is unproductive
-

5. Output

Final vectors (z^*) are:

- Relation-consistent and semantically coherent
- Optionally refined through DM perturbations if novel paths were explored
- Decoded by the Decoder into human-readable outputs

Example:

Query: “Apply evolutionary principles to optimize neural networks”

- EC starts with “evolution → neural network optimization”
- If novelty drops, DM explores alternative analogies: population-based methods, mutation-based optimization, meta-learning strategies
- EC resumes to verify alignment and produce coherent outputs:
 - “Evolutionary algorithms for network optimization”
 - “Adaptive selection in hyperparameter tuning”

Traversal in Default Mode (DM) with Dynamic Switching

Default Mode emphasizes **exploratory, creative reasoning**, but may **switch to EC** for refinement when salience detects emerging coherence or a relational vector arises.

1. Initial Position

Traversal begins at one or more entity embeddings (z_0) projected from the query.

- If no explicit relation is provided, DM relies on local manifold geometry, novelty gradients, and structural perturbations.

- Candidate vectors are stored in Working Memory for later refinement.
-

2. Update Rule

At each step (t):

$$[\begin{aligned} z_{t+1} &= z_t + \eta \left(\lambda_2 \nabla_z C(z_t) + \lambda_3 \nabla_z N(z_t) + \lambda_4 \right. \\ &\quad \left. P(z_t) \right) \end{aligned}]$$

Where:

- (λ_2) = coherence weight
 - (λ_3) = novelty weight (emphasized)
 - (λ_4) = structural perturbation weight
 - $(P(z_t))$ = structural perturbation along learned axes
-

3. Structural Perturbation Mechanism

$$[\begin{aligned} z' &= z_t + \sum_i \tau_i a_i + \epsilon R(z_t) \end{aligned}]$$

- $(\tau_i \sim \text{mathcal{N}}(0, \sigma^2))$
- $(R(z_t))$ = small random rotation in local tangent space
- (ϵ) = exploration magnitude

This mechanism recombines known ideas to discover novel conceptual configurations.

4. Dynamic Traversal Behavior

1. Primary DM behavior:

- Explore manifold broadly for new ideas
- Balance novelty ($(\nabla_z N)$) with minimal coherence checks

2. Salience-based switching:

- If a latent path exhibits sufficient coherence or a relation vector emerges, DM **triggers a switch to EC**
- EC refines outputs, ensuring relation-consistency and semantic alignment
- Working Memory retains all DM-generated candidates for EC to select from

3. Return to DM:

- EC can return control to DM if novelty decreases or additional creative paths are desired
- This allows DM to continue exploring directions previously untraveled

5. Termination Criteria

- Novelty plateau: $(|\nabla_t N(z_t)| < \epsilon_N)$
- Maximum step limit reached
- Salience/Verifier module decides further exploration is unnecessary

6. Output

Each final latent vector (z^*) represents:

- Semantically coherent new ideas
 - Novelty-enhanced candidates
 - Stored in Working Memory with coherence and novelty scores
 - Decoded into outputs such as:
 - “Bio-inspired chemical storage”
 - “Phase-change material batteries”
 - “Wireless energy transfer networks”
-

7. Summary of Dynamic Switching

- **Hybrid control:** intent-based initialization + salience-driven mode switching
- **DM ↔ EC loop:** the system explores, refines, and revisits ideas without losing prior paths
- **Working Memory & Verifier:** ensure candidate ideas are tracked, evaluated, and pruned if incoherent
- **Salience Network:** continuously monitors novelty and coherence to trigger mode switches, emulating human attention and focus

Here’s a patent-style writeup of the **Interaction of Traversal, Working Memory, and Verifier Module** that explicitly describes the **dynamic DM ↔ EC loop** with salience monitoring:

Interaction of Traversal, Working Memory, and Verifier Module

This embodiment describes how the system modules—Traversal (DM and EC), Working Memory, Verifier, and Saliency Network—interact to produce coherent, novel ideas while maintaining the ability to dynamically switch modes and revisit prior paths.

1. Working Memory

- **Purpose:** Temporarily stores candidate latent vectors, traversal paths, coherence metrics, novelty scores, and structural perturbations.
 - **Role in Dynamic Traversal:**
 1. **Records DM-generated ideas:** Each vector (z_t^{DM}) and its associated metrics ((C_t, N_t)) are stored.
 2. **Tracks EC refinements:** As EC traverses relation-aligned paths, each refinement step is logged.
 3. **Facilitates return to DM:** Previously explored DM paths remain accessible, allowing the system to resume creative exploration without reinitialization.
-

2. Verifier Module

- **Purpose:** Ensures semantic and relational consistency of candidate vectors.
- **Functions:**
 1. **Coherence pruning:** Removes or flags latent vectors that violate semantic consistency or coherence thresholds.
 2. **Relation validation:** Confirms that EC traversal aligns with the intended relational vector or query semantics.

3. **Working Memory pruning:** Deletes or marks incoherent DM candidates to prevent unproductive loops, while leaving promising vectors intact.
- **Interaction with Saliency:** Verifier feedback influences mode switching. If too many candidates are pruned or coherence drops, the Saliency Network may switch traversal from EC back to DM for additional exploration.
-

3. Saliency Network

- **Purpose:** Monitors novelty ((N_t)) and coherence ((C_t)) metrics to control adaptive mode switching.
 - **Dynamic Role:**
 1. **DM $\rightarrow$ EC switch:** Triggered when a DM path exhibits sufficient coherence or a latent relation vector emerges.
 2. **EC $\rightarrow$ DM switch:** Triggered when EC traversal stagnates, novelty drops below (τ_N) , or local minima prevent further relational alignment.
 3. **Continuous supervision:** Saliency constantly evaluates candidate paths in Working Memory, ensuring that mode switching preserves both creativity and relation consistency.
-

4. Traversal Module

- **Default Mode (DM):**
 - Generates diverse candidate vectors using structural perturbation ($(P(z_t))$) and novelty gradients ($(\nabla_z N)$).
 - Writes all candidates to Working Memory for potential refinement.

- **Executive Control (EC):**
 - Refines DM or direct-query candidates along relational vectors.
 - Monitors coherence ($(\nabla_z C)$) and applies Verifier pruning.
 - Can temporarily return control to DM if Saliency indicates insufficient novelty.
-

5. Dynamic Traversal Loop

1. Initialization:

- Query analyzed to detect relation vector presence.
- Mode initialized: DM if no explicit relation, EC if relation detected.

2. Traversal Steps:

- At each step, the active concept vector is updated according to mode-specific rules.
- DM emphasizes novelty + structural perturbation.
- EC emphasizes coherence + relational alignment.

3. Saliency Evaluation:

- Coherence ((C_t)) and novelty ((N_t)) monitored continuously.
- Working Memory stores all active and candidate vectors.

4. Verifier Check:

- Prunes incoherent candidates from Working Memory.
- Confirms relational consistency for EC candidates.

5. **Mode Switching (Adaptive Loop):**

- If DM produces a candidate with high coherence, Saliency triggers EC to refine.
- If EC stagnates, Saliency triggers DM to explore alternative directions, resuming from previously stored vectors.
- Loop continues until termination criteria are met (goal similarity, novelty plateau, max steps).

6. **Termination:**

- Final candidates in Working Memory are verified and decoded.
- Outputs include novel, semantically coherent ideas refined by EC, yet preserving creative diversity from DM.

6. **Example of Dynamic Loop**

Query: “Generate new ideas for sustainable urban transportation”

1. **DM Start:**

- Explores latent space with novelty gradients → “solar-powered scooters,” “adaptive bike lanes,” “modular transit pods.”
- Candidates stored in Working Memory.

2. **Saliency Trigger EC:**

- Coherence rises for “adaptive bike lanes.”
- EC takes over, refining relational alignment: “integrates real-time traffic optimization with bike lane routing.”

3. Stagnation Detected:

- EC cannot generate additional improvements; novelty drops.
- Salience switches back to DM to explore alternate directions: “autonomous micro-shuttles,” “dynamic pedestrian-prioritized streets.”

4. Loop Completes:

- Working Memory contains all candidate ideas.
- Verifier removes incoherent entries.
- Decoder outputs refined, creative, and verified set of solutions.

7. Advantages of This Architecture

- Preserves **all prior DM explorations**, avoiding premature loss of creative paths.
- Ensures **semantic validity and relational alignment** through Verifier feedback.
- Allows **adaptive back-and-forth switching** to balance novelty and coherence.
- Mirrors human cognition: exploratory creativity followed by evaluative refinement, continuously iterating without discarding previous insights.

Extended Cognitive Features

The following are extensions added to the basic core of the system described above in order to increase functionality and general intelligence behavior. However they are not **required** for the core mechanism to work.

1. Goal Component (Self-Modeling and Self-Set Goals)

The Goal Module governs the system's ability to reason about its own reasoning processes and to autonomously set new objectives. This component allows the system to move from query-directed reactive reasoning to self-driven curiosity and learning on its own designated terms. It consists of two integrated subsystems: a **self-modeling component** and a **self-generated goal component**.

(a) Self-Modeling:

The system continuously monitors its internal cognitive processes during reasoning episodes, held within the working memory — including which latent axes were traversed, which analogical paths succeeded or failed, and how verification outcomes evolved over time.

These traces are stored in a working memory structure, enabling the system to build a *model of its own reasoning strategies*.

Over multiple episodes, this model captures patterns in how analogies are formed and refined — allowing the system to improve its efficiency, generalization, and analogical precision across diverse tasks. The self-modeling module can be chosen to be run manually by the user or

(b) Self-Set Goals:

When operating in open-ended or **Default Mode**, the system can autonomously generate new goals by detecting regions of **low knowledge density** or **high novelty potential** within the latent space.

This process implements curiosity-driven exploration using a reward-like objective function:

$$\begin{aligned} &[\\ &G = \arg\max_z [\text{Novelty}(z) - \lambda, \text{Uncertainty}(z)] \\ &] \end{aligned}$$

where (Novelty(z)) measures conceptual distinctiveness within the latent geometry, and (Uncertainty(z)) quantifies the system's confidence in its current representations.

By optimizing this balance, the system selectively explores conceptual areas that are both *new* and *meaningful*, leading to self-generated questions, hypotheses, or ideas without explicit user input — a hallmark of general intelligence.

Reasoning:

This dual mechanism enables the system not only to execute externally provided tasks but also

to autonomously determine *what to think about next* and *how to improve its own reasoning strategies* over time. The Goal Module thereby transforms the architecture from a static reasoning engine into a self-improving, self-directed cognitive system.

Sample System Flow

The following section explains, with a sample query, how the flow of the system and all its components and mechanisms would work in tandem to handle it. The following numbers and vectors are only for demonstration purposes and do not represent an actual run of the system. This section also contains pseudocode for one embodiment of the system.

Sample user query:

“Can biological evolution principles improve machine learning optimization?”

(Incomplete section - coming soon)

Potential Applications & Embodiments

1. Scientific Hypothesis Generation

The system can propose new biological or physical hypotheses by analogically mapping concepts across domains (e.g., **cellular metabolism** ↔ **energy networks**).

- Inputs: literature embeddings + problem statement.
- Output: new testable hypotheses with explicit reasoning paths.

2. Autonomous Research Assistants

Embodied within a lab automation environment, the system could generate and evaluate experimental setups, self-correct hypotheses, and prioritize promising directions — effectively becoming a “scientific collaborator.”

3. Creative Design Systems

Used in engineering, architecture, or product design, it could analogically link materials science with natural forms (e.g., biomimicry).

It can propose **structurally novel yet functionally plausible** designs.

4. Knowledge Discovery in Enterprises

When applied to large organizational datasets, it can uncover hidden analogies between markets, products, and processes — suggesting innovative strategies or technologies.

5. Cognitive Simulation and AGI Research

The system provides a formal testbed for studying brain-like reasoning, creativity, and self-improving cognition.

Potential embodiment as part of a **cognitive architecture** in artificial general intelligence frameworks.

Claims (Broad Examples)

1. **A cognitive reasoning system** comprising:
 - a probabilistic variational autoencoder configured to encode domain concepts into a latent geometric space,
 - a dual-projection network configured to project queries both holistically and component-wise into the latent space,
 - a traversal module configured to explore the latent space via analogical alignment, and
 - a salience-driven mode controller that alternates between exploratory and goal-directed reasoning modes.
2. **The system of claim 1**, wherein a working memory module records the trajectory of latent traversals as tuples of semantic state, novelty, and coherence, enabling explainable reasoning.
3. **The system of claim 1**, wherein structural perturbations in latent space are applied to generate novel conceptual recombinations consistent with analogical structure.

4. **The system of claim 1**, further comprising a goal-generation module that autonomously identifies regions of low knowledge density or high novelty potential for exploration.
5. **A method of generating new ideas**, comprising:
 - encoding a query into a shared latent representation,
 - performing dual projection to align subcomponents to analogous domains,
 - traversing latent space using salience-guided exploration, and
 - decoding and verifying coherent, novel outputs.